BatFormer Details

Andrew Chen, March 2026, v2.0 - extended from Varun Desai's implementation of the Spacetimeformer (Grigsby et al. 2021)

The goal of this document is to explain some of the details of the BatFormer implementation that do not extend directly from the Spacetimeformer implementation.

Note on the structure of this package

The Spacetimeformer implementation lives in the bats repo at /bats/bats_transformer/spacetimeformer. The important files are as follows:

```
None
| spacetimeformer
| - | spacetimeformer
| - | - | spacetimeformer_model
| - | - | - | nn
| - | - | - | - model.py
| - | - | - spacetimeformer_model.py
| - | - forecaster.py
| - | - quantile_loss.py
```

model.py contains the architecture of the model itself (think the encoders, decoders, attention, etc.)

- Class: Spacetimeformer

forecaster.py is a base class that contains methods/properties related to the training of the model.

- Class: Forecaster

spacetimeformer_model.py inherits forecaster.py and creates an instance of the model from model.py.

- Class: Spacetimeformer_Forecaster
- Class: Spacetimeformer_Quantized_Forecaster

quantile_loss.py is a new file/class that implements a quantile loss function to train the model on.

- Class: QuantileLoss

Quantile Loss

The Spacetimeformer is originally trained on one of a few preset loss functions: MSE, MAE, or SMAPE. Independent research has shown that quantile loss may be a better loss function for our use case, where we model data with an uncertain amount of noise and where there may be trends at the tails of distributions in addition to at the means. The quantile loss is initialized by a list of quantiles that gets passed to the Spacetimeformer_Forecaster. The quantiles I used for training were [0.05, 0.5, 0.95].

Currently, the use of quantile loss and the values of the quantiles are hard-coded into /bats/bats_transformer/train.py. Future improvements to this should set these as arguments that can be used by train_bats_transformer.py.

Quantized Model

Because the Spacetimeformer_Forecaster is a regression model, there is no measure of confidence that the model returns. This is why, in identifying idioms, we train an ensemble of models and then calculate confidence/uncertainty from the ensemble of predictions. This is, however, only a proxy and also highly inefficient, so one solution is to use a quantized model that converts the regression task into a classification task.

For this reason, I drafted the Spacetimeformer_Quantized_Forecaster which extends the Spacetimeformer_Forecaster but changes the forward and loss step functions to fit the quantized classification task.

The way the quantized Spacetimeformer works is by binning each of the predicted measures into a set number of bins (either by splitting the range uniformly or by splitting the distribution by quantile), which then serve as the classes for classification.

Note: this class is just a draft and was not worked out very thoroughly for correctness. It may work fine, but a more careful pair of eyes should revise the code.

Command-line arguments

--n_bins N_BINS
    Number of bins for output quantization
--quantization_method QUANTIZATION_METHOD
    Method to compute quantization bins

Shuffling the data

Shuffling the training data is obviously an important debiasing strategy for most machine learning models. It also allows for techniques like cross-validation.

So I added a command-line argument for /bats/bats_transformer/train.py and /bats/bats_transformer/test.py that shuffle the dataset defined by the BatsCSVDataset(withMetadata) that shuffles the data.

But right now, it shuffles all of the data freely, meaning that different models trained with different random seeds will end up with different test sets. This didn't work with the way we used an ensemble of models to calculate confidence, so I ended up not using the shuffle flag when training. But if the next step is to use the quantized Spacetimeformer, then it may make sense to shuffle the data again.